



BITS Pilani

Modern Database Systems

Prof. Ashish Narang
CSIS



BITS Pilani



<SSZG507, Modern Database Systems>

Lecture No. 3

- Need for Distributed Databases
- Centralized vs Distributed Database Systems
- Distributed Database Architecture
- Types of Distributed Database Systems
- Challenges in Distributed Database Systems
- Data Partitioning and Fragmentation
- Data Allocation
- Data Replication
- Comparison of Replication Strategies

Need for Distributed Databases



- Modern applications may operate across **multiple geographical locations and computing sites**.
- A **distributed computing system** consists of multiple interconnected sites or nodes that cooperate to perform tasks.
- Instead of keeping all data and processing at a single location, data and computation can be distributed across networked nodes.
- Distributed data management can provide:
 - **Improved availability and reliability**
 - **Better performance through data localization**
 - **Support for geographically distributed applications**
 - **Easier expansion and scalability**

These requirements motivate the move from a **centralized database system** to a **Distributed Database System (DDBS)**.

What is a Distributed Database?



Distributed Database (DDB)

- A collection of **multiple logically interrelated databases** distributed over a **computer network**.
- Data may be physically stored at **different sites or nodes**, but it is logically related.
- Each site may perform **local processing** and may also participate in **global transactions**.
- To users, the distributed data should ideally appear as a **single integrated database**.

Distributed Database Management System (DDBMS)

- Software that manages the distributed database and coordinates access across different sites.
- Provides mechanisms for **data access, communication, transaction management, and coordination** among participating sites.

Characteristics of a Distributed Database System



A Distributed Database System typically has the following characteristics:

- **Data Distribution** – Data is physically stored across multiple sites connected through a network.
- **Logical Integration** – Distributed data forms a logically related and integrated database.
- **Local Autonomy** – Individual sites may have some control over their local data and operations.
- **Network Communication** – Sites communicate and exchange data through a communication network.
- **Distributed Processing** – Queries and transactions may involve data stored at multiple sites.
- **Distribution Transparency** – Users should ideally access data without needing to know its physical location.
- **Reliability & Availability** – Distribution can allow continued operation even when some sites or communication links fail.

Advantages of Distributed Databases



Distributed database systems offer several advantages:

- **Improved Reliability and Availability**
Failure of one site does not necessarily make the entire database inaccessible.
- **Improved Performance**
Data can be stored close to where it is most frequently used, reducing data communication and access time.
- **Local Control of Data**
Individual sites can retain a degree of control over their locally stored data.
- **Modular Growth**
New sites and computing resources can be added as organizational requirements grow.
- **Reduced Communication Overhead**
Frequently accessed data can be processed locally instead of repeatedly accessing a remote central database.
- **Support for Distributed Organizations**
The database structure can naturally reflect organizations operating across multiple geographical locations.

Centralized vs Distributed Database Systems



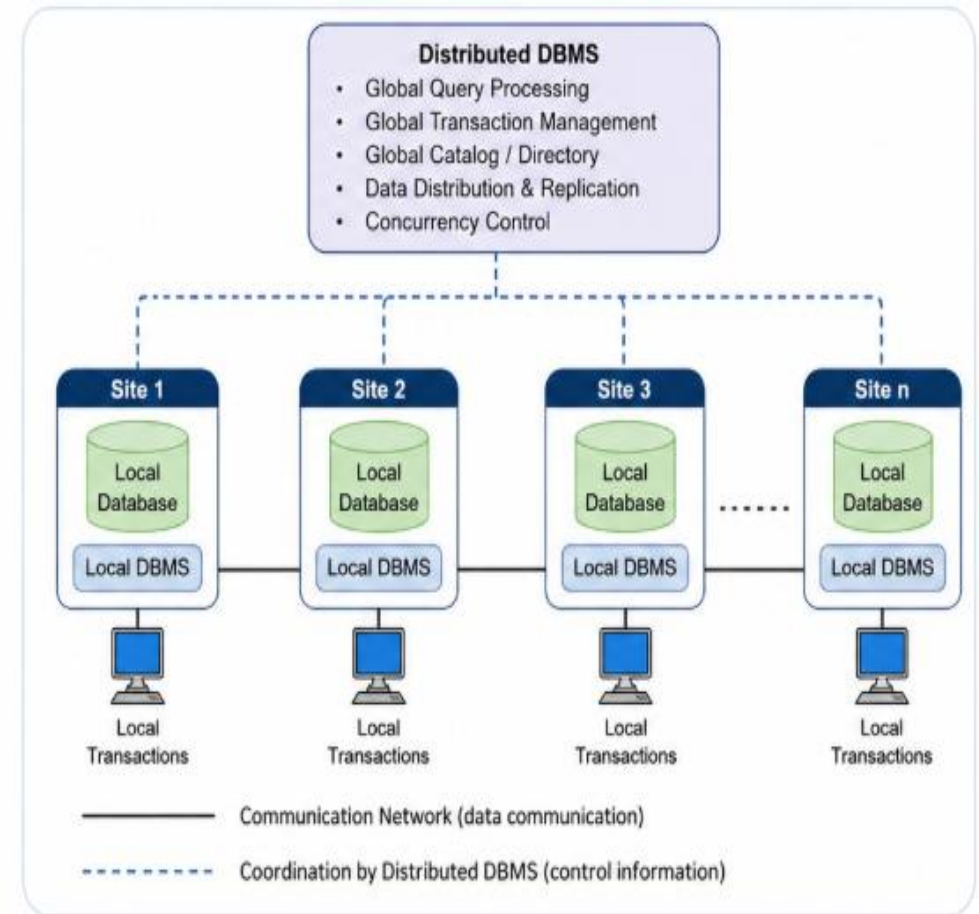
Centralized Database System	Distributed Database System
Data primarily stored at a single site	Data stored across multiple sites
Centralized processing and control	Processing may occur at multiple sites
Simpler administration and coordination	Requires coordination among distributed sites
Central site can become a single point of failure	Failure of one site may allow other sites to continue
Scaling often requires increasing central resources	Supports growth by adding sites/resources
Remote users depend heavily on network access to central site	Data can be located closer to users
Lower distribution-management complexity	Higher communication and consistency complexity

Distributed Database Architecture: Basic View



A Distributed Database System consists of **multiple interconnected sites**, where each site may contain:

- **Local Database** – Stores data assigned to that site.
- **Local DBMS** – Manages local data and local transactions.
- **Communication Network** – Connects participating database sites.
- **Distributed DBMS Functions** – Coordinate access to data across multiple sites.
- **Local Transactions** – Access data stored at a single site.
- **Global Transactions** – Access data stored at more than one site.



Types of Distributed Database Systems



Distributed database systems can be classified based on the **similarity of DBMSs, data models, and degree of site autonomy**.

1. Homogeneous Distributed DBMS

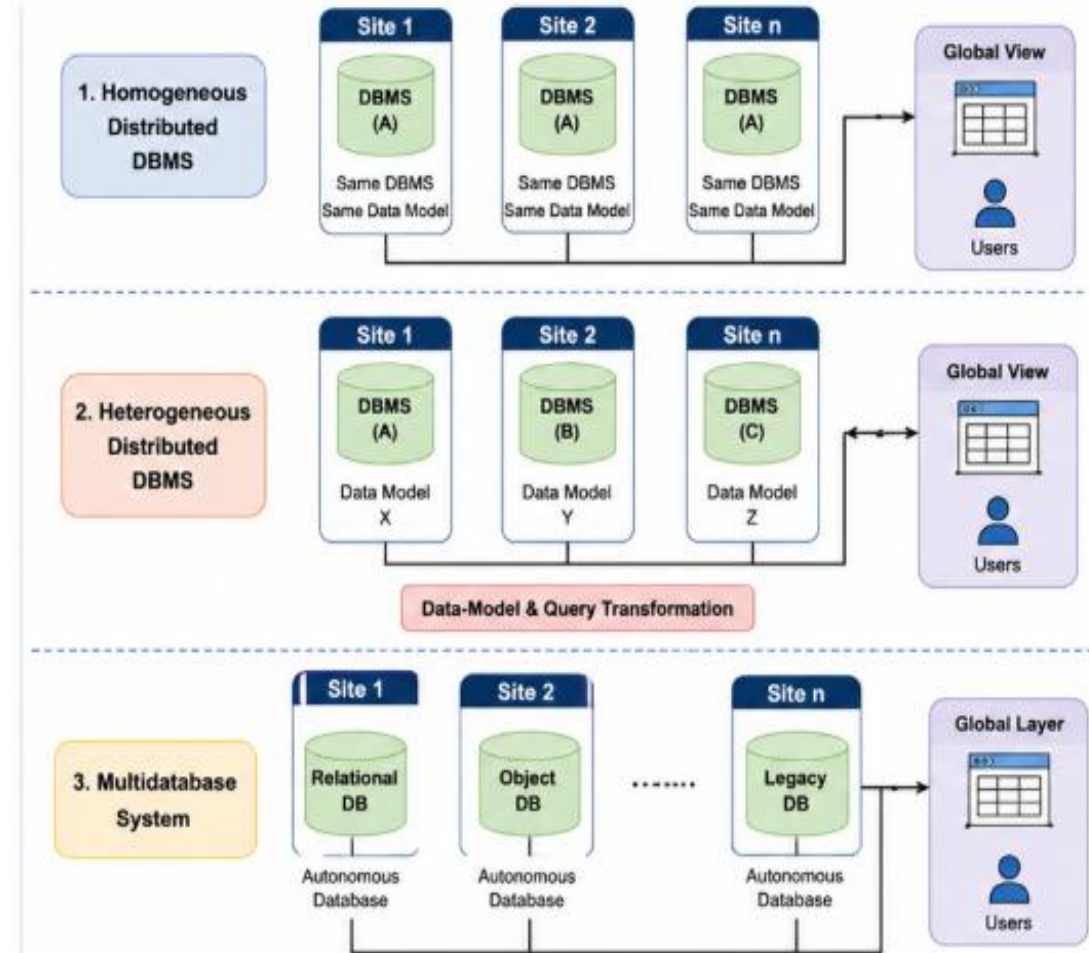
- All sites use the **same DBMS software**.
- Sites typically use the **same data model**.
- Integration and communication are relatively easier.
- Appears more naturally as a **single distributed database system**.

2. Heterogeneous Distributed DBMS

- Sites may use **different DBMS products**.
- Different sites may use **different data models or schemas**.
- Requires mechanisms for **data-model and query transformation**.
- Integration and coordination are more complex.

3. Multidatabase System

- Integrates multiple **pre-existing autonomous databases**.
- Individual databases may retain significant **local autonomy**.
- A global layer provides access across participating databases.



Challenges in Distributed Database Systems



Distributing data across multiple sites introduces several challenges:

- **Network Communication** – Data and control messages must travel between sites.
- **Site & Network Failures** – Individual nodes or communication links may become unavailable.
- **Data Consistency** – Multiple copies of data must remain appropriately synchronized.
- **Distributed Transaction Management** – Transactions involving multiple sites require coordination.
- **Concurrency Control** – Concurrent operations at different sites must preserve database correctness.
- **Distributed Query Processing** – Accessing data from multiple sites can increase communication and processing cost.
- **Security** – Data and communication must be protected across multiple locations.
- **System Complexity** – Administration, monitoring and recovery become more difficult than in centralized systems.

Data Partitioning in Distributed Databases



Data Partitioning (Fragmentation) divides a database relation into smaller **fragments** that can be stored at different sites.

Why Partition Data?

- Place data **closer to where it is frequently accessed**
- Reduce unnecessary **data communication**
- Enable **parallel processing** across sites
- Improve **locality of reference**
- Support distribution according to organizational requirements

Main Types of Fragmentation

- **Horizontal Fragmentation:** Divides a relation into subsets of **rows (tuples)**.
- **Vertical Fragmentation:** Divides a relation into subsets of **columns (attributes)**.
- **Mixed/Hybrid Fragmentation:** Combines horizontal and vertical fragmentation.

Horizontal Fragmentation



Horizontal Fragmentation divides a relation into subsets of **tuples (rows)** based on selection conditions.

How Does It Work?

- Each fragment contains a **subset of rows** from the original relation.
- Fragments generally retain the **same attributes** as the original relation.
- Selection conditions determine which tuples belong to each fragment.
- Different fragments can be allocated to **different sites**.

Consider:

EMPLOYEE (Emp_ID, Name, Department, Location, Salary)

Delhi Fragment

$EMP_{Delhi} = \sigma_{Location='Delhi'}(EMPLOYEE)$

Mumbai Fragment

$EMP_{Mumbai} = \sigma_{Location='Mumbai'}(EMPLOYEE)$

Reconstruction

The original relation can be reconstructed using

UNION:

$EMPLOYEE = EMP_{Delhi} \cup EMP_{Mumbai}$

Primary and Derived Horizontal Fragmentation



Horizontal fragmentation can be further classified into:

Primary Horizontal Fragmentation

- Performed using **predicates defined on the relation itself**.
- Tuples are grouped according to conditions on the relation's own attributes.
- **Example:**

$EMP_{Delhi} = \sigma_{Location='Delhi'}(EMPLOYEE)$

Derived Horizontal Fragmentation

- Fragmentation of one relation is based on the **fragmentation of another related relation**.
- Commonly used when two relations are connected through a relationship.
- Helps place **related data at the same site**.

Example:

- If DEPARTMENT is fragmented by location, EMPLOYEE can be fragmented according to the corresponding DEPARTMENT fragments.

Common Clauses Used with SELECT



Vertical Fragmentation divides a relation into subsets of **attributes (columns)**.

How Does It Work?

- Each fragment contains selected **attributes** of the original relation.
- Attributes frequently accessed together can be placed in the **same fragment**.
- The **primary key (or tuple identifier)** is included in each vertical fragment.
- Different fragments may be allocated to different sites based on their usage.

Consider:

EMPLOYEE (Emp_ID, Name, Department, Location, Salary)

Fragment 1 – General Information

$EMP1 = \pi_{Emp_ID, Name, Department}(EMPLOYEE)$

Fragment 2 – Payroll Information

$EMP2 = \pi_{Emp_ID, Salary, Contact}(EMPLOYEE)$

Reconstruction

The original relation can be reconstructed using a **JOIN** on the common key:

$EMPLOYEE = EMP1 \bowtie_{Emp_ID} EMP2$

Mixed (Hybrid) Fragmentation



Mixed Fragmentation combines **horizontal and vertical fragmentation** to distribute data according to application requirements.

How Does It Work?

A relation may be fragmented:

- **Horizontally first, then vertically**, or
- **Vertically first, then horizontally**

Example

Consider:

EMPLOYEE (Emp_ID, Name, Department, Location, Salary)

Step 1 – Horizontal Fragmentation by Location

- $EMP_{Delhi} = \sigma_{Location='Delhi'}(EMPLOYEE)$
- $EMP_{Mumbai} = \sigma_{Location='Mumbai'}(EMPLOYEE)$

Step 2 – Vertical Fragmentation of Delhi Data

- $EMP_{Delhi-General} = \pi_{Emp_ID, Name, Department}(EMP_{Delhi})$
- $EMP_{Delhi-Payroll} = \pi_{Emp_ID, Salary}(EMP_{Delhi})$

Data Allocation in Distributed Databases



- After fragmentation, the next decision is **where each fragment should be stored**.
- **Data Allocation** refers to assigning database fragments to different sites in a distributed system.
- **Allocation Strategies**
 - 1. Non-Replicated (Partitioned) Allocation**
 - Each fragment is stored at **only one site**.
 - No duplicate copies are maintained.
 - 2. Fully Replicated Allocation**
 - A complete copy of the database is stored at **every site**.
 - Provides high availability but increases update overhead.
 - 3. Partially Replicated Allocation**
 - Selected fragments are stored at **multiple sites**.
 - Other fragments may exist at only one site.
 - Provides a balance between availability and update cost.

Data Replication in Distributed Databases



Data Replication involves maintaining **multiple copies of the same data** at different sites in a distributed database system.

Why Replicate Data?

- **Improved Availability** – Data may remain accessible even if one site fails.
- **Improved Reliability** – Multiple copies reduce dependence on a single site.
- **Faster Read Access** – Users can access a nearby replica.
- **Reduced Communication Cost** – Frequently accessed data can be available locally.
- **Support for Parallel Reads** – Read requests can be served by different replicas.

Replication Trade-off



Benefits

- Higher availability
- Better read performance
- Greater fault tolerance

Costs

- Additional storage
- Increased update overhead
- Maintaining consistency among replicas

Degrees of Data Replication



Depending on the number of copies maintained, a distributed database can use different **degrees of replication**.

1. Fully Replicated Database

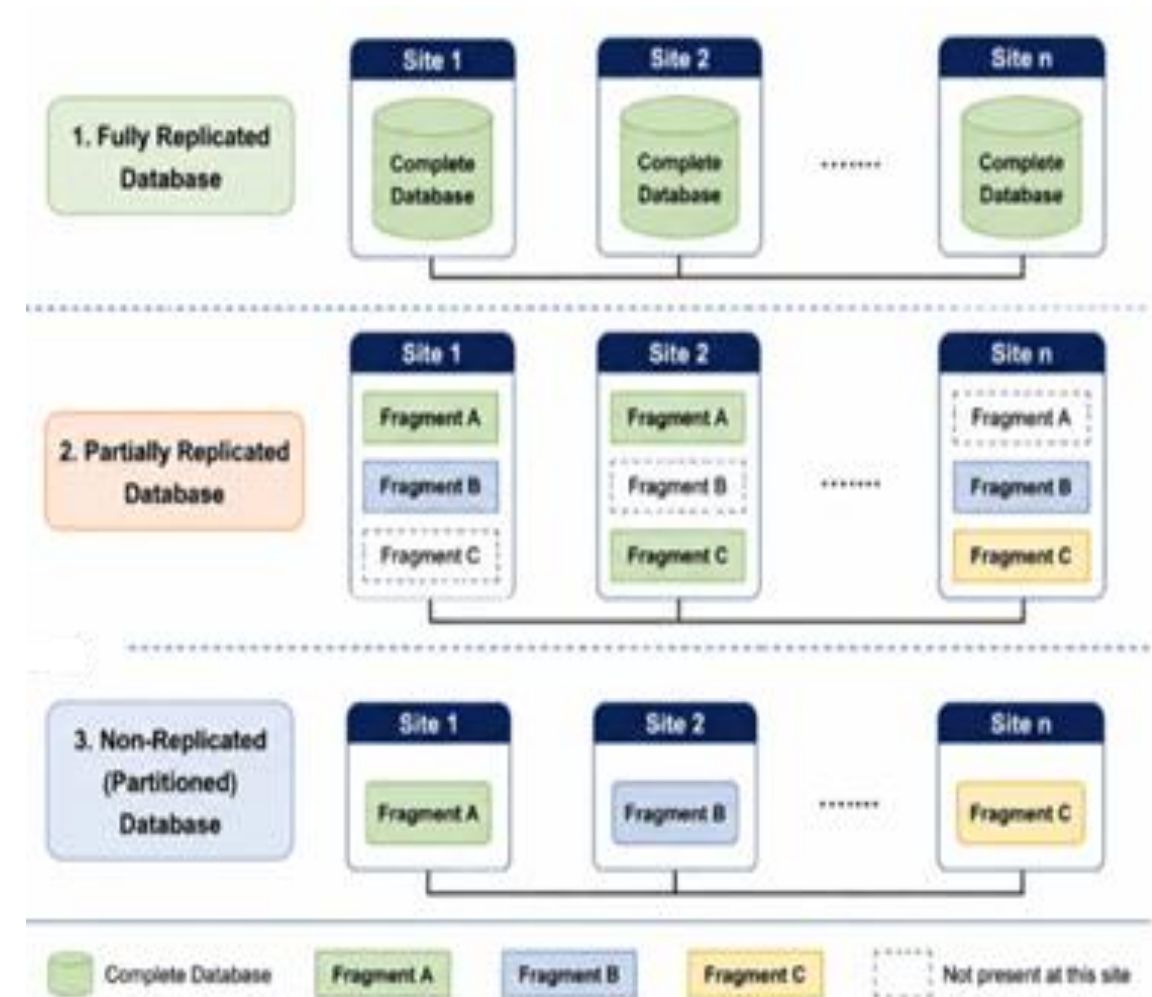
- A **complete copy of the database** is stored at every site.
- Provides high **availability and reliability**.
- Read operations can often be performed locally.
- Updates must be propagated to **all replicas**.

2. Partially Replicated Database

- Only **selected fragments** are replicated.
- Different fragments may have different numbers of copies.
- Balances **availability, performance, storage, and update cost**.

3. Non-Replicated (Partitioned) Database

- Each fragment is stored at **one site only**.
- Lower storage and update overhead.
- Availability depends on the site containing the required fragment.



Synchronous vs Asynchronous Replication



- When data is replicated, an important decision is **when updates are propagated to replicas**.

Synchronous Replication	Asynchronous Replication
Update is propagated before the operation is considered complete	Primary update can complete before replicas are updated
Replicas remain closely synchronized	Replicas may temporarily contain older data
Stronger consistency between copies	Temporary inconsistency may occur
Higher write latency	Lower write latency
More sensitive to slow/unavailable replicas	Better tolerance of network delays
Suitable when up-to-date replicas are critical	Suitable when performance and geographic distribution are important

Leader–Follower Replication



In **Leader–Follower Replication**, one replica is designated as the **Leader (Primary)** and other replicas act as **Followers (Replicas/Secondaries)**.

How It Works

- **Writes are sent to the Leader.**
- Leader updates its local database.
- Leader sends the changes to its **Followers**.
- Followers apply the changes to their local copies.
- **Reads** may be served by the leader or followers, depending on system configuration.

Characteristics

- Provides a **single point for coordinating writes**.
- Followers maintain copies of the leader's data.
- Replication may be **synchronous or asynchronous**.
- Followers can help **scale read workloads**.
- If the leader fails, a follower may be **promoted as the new leader**.
- Asynchronous followers may return **stale data** because of replication lag.

Multi-Leader Replication



How It Works

- Multiple nodes act as **Leaders**.
- Clients can write to different leaders.
- Each leader applies its **local writes**.
- Leaders replicate their changes to **other leaders**.
- Conflicting concurrent updates may require **conflict detection and resolution**.

Where is it Useful?

- **Multi-datacenter deployments**
- Applications with users distributed across **geographical regions**
- Environments requiring **local write processing**
- Systems that may need to continue accepting writes despite connectivity problems between sites

Major Challenge

Write Conflicts

Leader A: $X \rightarrow 100$

Leader B: $X \rightarrow 200$

Leaderless Replication



In **Leaderless Replication**, there is **no designated leader** for handling writes.

How It Works

- Clients can send writes to **multiple replicas directly**.
- A write is considered successful after a required number of replicas acknowledge it.
- Reads may also query **multiple replicas**.
- Differences between replicas may be detected and repaired.
- The system continues to operate without depending on a single leader.

Quorum-Based Approach

Let:

- **N** = Number of replicas
- **W** = Number of replicas that must acknowledge a write
- **R** = Number of replicas contacted for a read

A commonly discussed quorum condition is:

$$W+R>N$$

This creates an **overlap between the replicas involved in reads and writes**.

Comparing Replication Strategies



Aspect	Leader–Follower	Multi-Leader	Leaderless
Write coordination	Single leader	Multiple leaders	No designated leader
Writes accepted by	Leader	Any leader	Multiple replicas
Read scalability	High through followers	High	High
Write locality	Limited by leader location	High	High
Conflict possibility	Relatively lower	Higher	Higher
Failure handling	Leader failover required	Other leaders may continue	No leader failover required
Main challenge	Leader failure / replication lag	Write conflicts	Replica divergence / coordination
Typical use	General-purpose replication	Multi-region systems	Highly available distributed systems

- **Need for Distributed Databases**
 - ✓ Availability, performance, scalability and geographical distribution
- **Centralized vs Distributed Systems**
 - ✓ Distribution provides locality and availability at the cost of increased coordination
- **Distributed Database Architecture**
 - ✓ Multiple interconnected sites with local and global operations
 - ✓ Homogeneous, heterogeneous and multidatabase environments
- **Challenges in Distributed Systems**
 - ✓ Communication, failures, consistency, concurrency and coordination
- **Data Partitioning / Fragmentation**
 - ✓ Horizontal fragmentation
 - ✓ Primary & derived horizontal fragmentation
 - ✓ Vertical fragmentation
 - ✓ Mixed fragmentation
- **Data Allocation & Replication**
 - ✓ Non-replicated, partially replicated and fully replicated data
 - ✓ Synchronous vs asynchronous replication
 - ✓ Leader–Follower, Multi-Leader and Leaderless replication



Thank You